Innovators@DravidianCodeMix 2026: A Unified Transformer-Based Approach to Hate and Hope Speech Detection in Code-Mixed Dravidian Languages

Shudapreyaa R S ,Harish Sri Ragunath V, Jaiyanth T, Hari Prasath G

Innovators

Abstract

This paper presents the Innovators team's participation in DravidianCodeMix 2026 across three task settings: Tamil-English hate speech, Tulu hope speech, and Kannada hate speech. We use multilingual Transformer encoders with task-specific classification heads to accommodate the distinct annotation structures. The Tamil-English system employs a five-level hierarchical classifier based on XLM-R large; the Tulu system jointly predicts coarse- and fine-grained hope-speech labels using XLM-R large; and the Kannada system performs binary hate-speech classification using XLM-R base with weighted cross-entropy. Training uses supervised fine-tuning, AdamW optimization, learning-rate scheduling, gradient clipping, checkpoint selection, and GPU-based mixed-precision training where configured. On the official leaderboard, Innovators ranked second in Subtask 1 with an overall Macro-F1 of 0.4394, first in Subtask 2 with an accuracy of 0.5121 and Macro-F1 of 0.4348, and third in Subtask 3 with Macro-F1 of 0.7652. The results show that a shared multilingual foundation can be effectively adapted to heterogeneous hate- and hope-speech annotation schemes, while deeper hierarchical levels remain challenging.

Keywords

hate speech detection; hope speech detection; code-mixed text; Dravidian languages; multilingual Transformers; XLM-R; hierarchical classification; multi-task learning

1. Introduction

User-generated content in Dravidian languages frequently combines native scripts, English, transliterated forms, informal spelling, emojis, abbreviations, and culturally dependent expressions. Such code-mixed text creates challenges for automatic language understanding because semantically related messages can occur in multiple scripts and spelling variants, while short social-media comments may provide limited context. Hate-speech detection is further complicated by implicit insults, target-dependent meanings, and fine-grained distinctions among related categories.

DravidianCodeMix 2026 provides a common evaluation setting for hate and hope speech detection in Dravidian-language data. The Innovators team participated in all three task settings: hierarchical Tamil-English hate-speech classification, multi-task Tulu hope-speech classification, and binary Kannada hate-speech classification. Rather than using one identical output layer for every task, we retain a common multilingual Transformer foundation and adapt the prediction head to each annotation structure.

The main contributions of this working note are: (1) a unified XLM-R-based workflow covering all three shared-task settings; (2) task-specific heads for hierarchical, joint coarse/fine, and binary classification; (3) weighted training for the Kannada class imbalance; (4) validation with accuracy, Macro-F1, Weighted-F1, and joint scoring where applicable; (5) analysis of development behavior and implementation issues; and (6) a reproducible training and prediction pipeline released with the team's repository.

2. Task Description

2.1 Tamil-English Hate Speech

Subtask 1 addresses hate speech in Tamil-English code-mixed text. The data can contain Tamil in native script, English, Romanized Tamil, or combinations of these forms. The task uses a hierarchical annotation scheme with five prediction levels. The Innovators model predicts Level 1 through Level 5 labels using a shared XLM-R representation and five task-specific linear heads. The head sizes in the implemented classifier are 3, 6, 3, 32, and 7 classes, respectively.

2.2 Tulu Hope Speech

Subtask 2 focuses on hope speech in code-mixed Tulu. The system predicts a coarse-grained tone label and a fine-grained hope-type label. The implemented model contains three coarse classes and five fine classes and uses a shared Transformer representation followed by separate coarse and fine prediction heads. The two objectives are optimized jointly, allowing contextual information to be shared between the label spaces.

2.3 Kannada Hate Speech

Subtask 3 is a binary hate-speech classification task for Kannada-English content. The training set used in the documented experiment contains 3,040 examples, including 1,687 instances of class 0 and 1,353 of class 1. The validation set contains 500

examples, with 288 class-0 and 212 class-1 instances. Because macro-F1 is sensitive to minority-class performance, the training pipeline uses weighted cross-entropy.

3. Dataset and Preprocessing

The task data are loaded from CSV files. Where required, unnecessary index columns are removed and target labels are converted to integer identifiers using LabelEncoder objects fitted only on the training data. The fitted encoders are saved so that model outputs can be mapped back to the original labels during prediction.

Input comments are converted to strings and tokenized with the tokenizer corresponding to the selected XLM-R checkpoint. Sequences are truncated to the configured maximum length and padded to form batches. Attention masks distinguish actual tokens from padding. The preprocessing intentionally avoids aggressive normalization so that script information, transliteration patterns, punctuation, emojis, and informal spelling remain available to the Transformer.

For reproducibility, the prediction stage uses the same tokenizer, sequence configuration, encoder architecture, classification-head structure, and saved checkpoint used during training. Integer predictions are converted back to the original labels with the saved encoders before producing the submission file.

4. Methodology

4.1 Multilingual Transformer Encoder

XLM-RoBERTa (XLM-R) is used as the main multilingual representation model. XLM-R was pretrained across many languages and is designed to transfer contextual representations across multilingual and low-resource settings. We use XLM-R large for the Tamil-English and Tulu experiments and XLM-R base for the Kannada experiment. The encoders are fine-tuned with supervised task losses rather than used only as frozen feature extractors.

4.2 Task-Specific Classification Heads

The Tamil-English model applies dropout to the pooled first-token representation and feeds the representation through five hierarchical heads. Each subsequent head receives the shared representation together with the preceding head's logits. The Tulu model applies a shared feed-forward transformation followed by independent coarse and fine heads. The Kannada model uses dropout, a linear projection, ReLU activation, a second dropout layer, and a final two-class classifier.

4.3 Training and Optimization

All three systems are trained with AdamW. The training workflows include learning-rate scheduling, gradient clipping, checkpointing, and early stopping or best-checkpoint selection based on validation performance. GPU acceleration is used for the reported runs. Automatic mixed precision is used where configured to reduce memory usage and improve throughput.

For Kannada, weighted cross-entropy is used to reduce the effect of class imbalance. For Tulu, the coarse and fine losses are optimized jointly. For Tamil-English, the model simultaneously learns all five hierarchical outputs. During development, validation predictions are accumulated over the complete validation set before computing metrics.

4.4 Experimental Configuration

Subtask	Model	Training data	Validation data	Epochs	Main configuration
1	XLM-R large	4,255	532	8	5 hierarchical heads: 3/6/3/32/7
2	XLM-R large	3,513	753	10	Shared layer + 3-class coarse + 5-class fine
3	XLM-R base	3,040	500	10	Max length 128; batch 8; AdamW; weighted CE

Table 1: Experimental configuration and development-set sizes used for the reported runs.

5. Development Results

Development results were used for model selection and debugging and are reported separately from the official leaderboard results. Figure 1 summarizes the validation progression of the three systems using the task-specific primary validation score. For Subtask 1, the plotted score is the mean Macro-F1 across the five levels; for Subtask 2, it is the reported joint score; and for Subtask 3, it is Macro-F1.

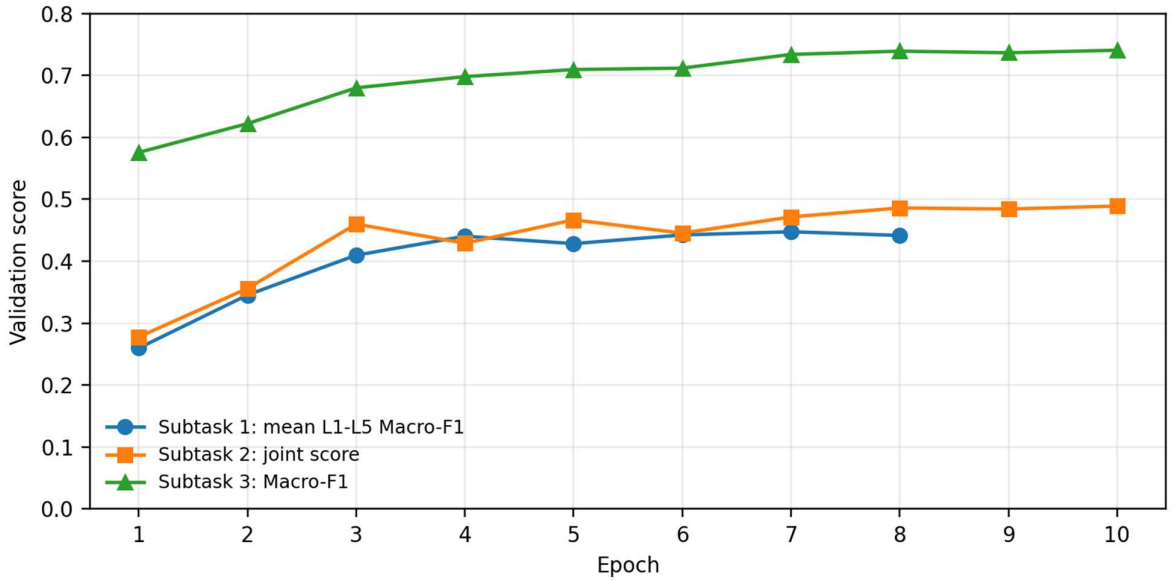


Figure 1: Validation performance across training epochs for the three subtasks.

5.1 Tamil-English Development Results

The Tamil-English model improved steadily through training, particularly at Levels 1–3. The best Level-4 validation Macro-F1 was 0.1162 at epoch 7, with Level-4 accuracy of 0.5075. The final epoch did not improve the Level-4 selection criterion, so the epoch-7 checkpoint was retained for that development criterion.

Level	Best validation accuracy	Best validation Macro-F1	Epoch
L1	0.8195	0.5468	3
L2	0.6974	0.6118	7
L3	0.6936	0.6732	6
L4	0.5075	0.1162	7
L5	0.8872	0.3440	4

Table 2: Best development metrics observed for each Tamil-English hierarchy level.

5.2 Tulu Development Results

The Tulu system improved from a joint score of 0.2779 in epoch 1 to 0.4892 in epoch 10. The best fine-grained validation Macro-F1 was 0.4356 at epoch 10, while fine-grained accuracy was 0.4900. The coarse classifier reached its highest validation accuracy of 0.6388 at epoch 9 and its highest Macro-F1 of 0.5428 at epoch 10.

Epoch	Coarse Acc.	Coarse Macro-F1	Fine Acc.	Fine Macro-F1	Joint Score
1	0.4449	0.3421	0.2643	0.2137	0.2779
2	0.5432	0.4334	0.3825	0.2791	0.3562
3	0.6069	0.5188	0.4714	0.4014	0.4601
4	0.5976	0.4972	0.4568	0.3607	0.4290
5	0.6215	0.5215	0.4754	0.4123	0.4669
6	0.6375	0.5159	0.4449	0.3750	0.4454
7	0.6189	0.5307	0.4608	0.4125	0.4716
8	0.6308	0.5418	0.4768	0.4303	0.4861
9	0.6388	0.5353	0.4980	0.4333	0.4843
10	0.6321	0.5428	0.4900	0.4356	0.4892

Table 3: Complete Tulu validation trajectory reported by the training run.

5.3 Kannada Development Results

The corrected Kannada run showed consistent improvement from 0.5758 Macro-F1 in epoch 1 to 0.7407 in epoch 10. Accuracy reached 0.7480 at epochs 8 and 10. The best recorded validation Macro-F1 was 0.7407 in epoch 10.

Epoch	Accuracy	Macro-F1
1	0.5760	0.5758
2	0.6280	0.6222
3	0.6800	0.6800
4	0.7040	0.6981
5	0.7100	0.7095
6	0.7120	0.7118
7	0.7420	0.7339
8	0.7480	0.7392
9	0.7440	0.7366
10	0.7480	0.7407

Table 4: Kannada validation accuracy and Macro-F1 by epoch.

6. Official Results

The official leaderboard results are the primary competition results and are distinct from the development metrics above. Innovators ranked in the top three in every submitted task: second in Tamil-English hate speech, first in Tulu hope speech, and third in Kannada hate speech.

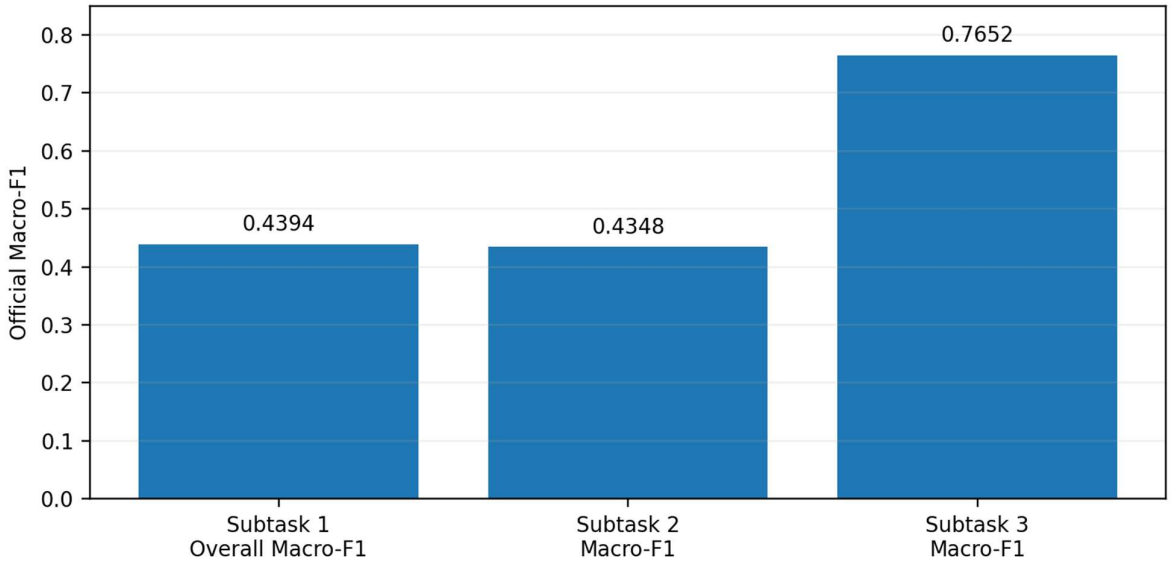


Figure 2: Official Macro-F1 achieved by Innovators across the three DravidianCodeMix 2026 subtasks.

Subtask	Language / Setting	Accuracy	Macro-F1	Official result
1	Tamil-English Hate Speech	—	0.4394	Rank 2
2	Tulu Hope Speech	0.5121	0.4348	Rank 1
3	Kannada Hate Speech	—	0.7652	Rank 3

Table 5: Official leaderboard performance for the Innovators submissions.

6.1 Subtask 1 Official Analysis

Subtask 1 is ranked using the mean Macro-F1 over Levels 1–5. Innovators obtained an overall Macro-F1 of 0.4394, placing second among the reported submissions. The official level-wise scores were 0.5403 for Level 1, 0.5231 for Level 2, 0.6729 for Level 3, 0.1048 for Level 4, and 0.3560 for Level 5.

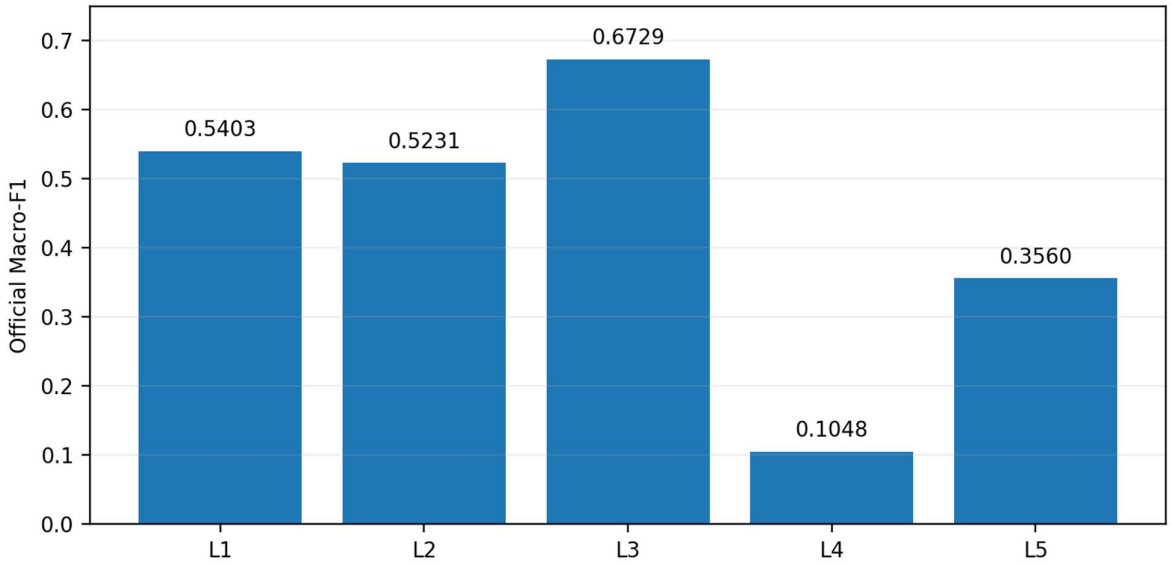


Figure 3: Official level-wise Macro-F1 for the Innovators Tamil-English system.

Team	L1 F1	L2 F1	L3 F1	L4 F1	L5 F1	Overall Macro-F1	Rank
HateLens	.5428	.5309	.7110	.1521	.3648	.4603	1
Innovators	.5403	.5231	.6729	.1048	.3560	.4394	2
KEC IJK	.5331	.4803	.6549	.1335	.2072	.4018	3
SIST_NLP	.5389	.4861	.6603	.0656	.2408	.3984	4
Dravidian Dropout	.5272	.4719	.6325	.0909	.2604	.3966	5
Vectra	.5137	.4711	.6322	.0713	.2403	.3857	6
NexaNLP	.5342	.4020	.6524	.0638	.2462	.3797	7
NLPFusion	.5085	.2974	.5752	.0433	.1581	.3165	8
MMIP Lab	.5185	.1759	.6083	.0435	.1405	.2973	9
TokenX	.3049	.1607	.2345	.0000	.1935	.1787	10

Table 6: Subtask 1 official rank list supplied by the shared-task leaderboard.

The main limitation is Level 4, whose Macro-F1 of 0.1048 is substantially below the other levels. This suggests that the deeper annotation distinctions are considerably harder for the current representation and head design. Levels 1–3 are comparatively strong, with Level 3 reaching 0.6729.

6.2 Subtask 2 Official Analysis

Innovators achieved first place in Subtask 2 with an official accuracy of 0.5121 and Macro-F1 of 0.4348. The leaderboard also reports the component evaluation for the coarse and fine settings. For the Innovators run, the coarse component records 0.3495 accuracy and 0.3019 Macro-F1, while the reported overall submitted-run score is 0.4348 Macro-F1.

Rank	Team	Run	Accuracy	Macro P	Macro R	Macro F1	Weighted F1
1	Innovators	Run1	.5121	.4386	.4377	.4348	.5210
2	MMIP Lab	Run1	.4785	.4063	.4040	.3981	.4806
3	TokenX	Run1	.4651	.4160	.4084	.3948	.4776
4	NexaNLP	Run1	.4530	.4231	.4049	.3694	.4501
5	TokenX	Run2	.4462	.3964	.3891	.3629	.4455
6	Dravidian Dropout	Run1	.4140	.3969	.4006	.3629	.4267

Table 7: Official Subtask 2 overall leaderboard supplied for the Innovators submission.

6.3 Subtask 3 Official Analysis

Innovators achieved third place in the Kannada hate-speech task with an official Macro-F1 of 0.7652. The submitted run recorded precision of 0.7653 and recall of 0.7679. The result is stronger than the best development Macro-F1 of 0.7407, showing that the model generalized well to the official evaluation set.

Team	Run	Precision	Recall	F1 Score	Rank
TokenX	Run1	.7782	.7790	.7720	1
TokenX	Run2	.7782	.7790	.7720	1
Coders	Run1	.7679	.7667	.7672	2
Innovators	Run1	.7653	.7679	.7652	3
NexaNLP	Run1	.7616	.7624	.7620	4
NLPFusion	Run2	.7478	.7503	.7473	5
TokenX	Run3	.7442	.7438	.7360	6
NLPFusion	Run3	.7329	.7347	.7298	7
NLPFusion	Run1	.6658	.5758	.5289	8
MMIP-LAbs	Run1	.5000	.5000	.4995	9

Table 8: Official Subtask 3 leaderboard supplied for the Innovators submission.

7. Error Analysis and Observations

The development experiments revealed that the three tasks have different error sources. Tamil-English contains code switching, transliteration, spelling variation, sarcasm, and implicit abuse. Tulu combines low-resource language characteristics with two related but distinct label spaces. Kannada contains short comments, class imbalance, lexical overlap, and context-dependent expressions. These factors can create confusion among harmful, neutral, supportive, and fine-grained categories.

The most visible Tamil-English weakness is the Level-4 hierarchy, where Macro-F1 remains close to 0.10 despite substantially higher performance at earlier levels. A deeper hierarchical classifier may therefore benefit from label-aware sampling, task-specific loss weighting, or conditioning later predictions on the discrete parent labels rather than only their logits.

The Tulu results show that joint modelling is viable but that the fine-grained task remains difficult. The fine Macro-F1 improves from 0.2137 to 0.4356 over the reported run. The gap between coarse and fine performance indicates that the broader tone can be identified more reliably than the precise hope type.

The Kannada experiments also highlighted the importance of checking prediction distributions. An earlier development run showed collapse toward one class. Inspecting logits, label distributions, optimizer updates, training-loop control flow, and complete validation accumulation helped identify and correct the behavior. The corrected run reached 0.7407 validation Macro-F1 and the submitted system achieved 0.7652 official Macro-F1.

7.1 Implementation Lessons

Two implementation checks were particularly important. First, an unintended break inside a training loop can terminate an epoch after a single batch and prevent the intended number of updates. Second, when an explicit batch index is needed, DataLoader iteration should use enumerate(train_loader); otherwise the batch object itself is returned rather than a step-and-batch pair. Validation predictions and targets should be accumulated over the complete validation set before metrics are calculated.

8. Prediction and Submission Pipeline

After training, the best checkpoint and task-specific label encoders are retained. The prediction scripts load the same tokenizer and model architecture used for training, tokenize each test comment with the configured maximum length, and generate predictions using the highest output logit. For hierarchical Tamil-English classification, each of the five outputs is decoded independently using the saved encoder for that level.

Before producing a final submission archive, the pipeline should verify that the number of predictions equals the number of test instances, every required identifier occurs exactly once, all predicted labels belong to the permitted label set, and no training or validation records are included. The repository contains task-specific training and prediction scripts for all three subtasks.

9. Reproducibility

The implementation uses Python, PyTorch, Hugging Face Transformers, pandas, NumPy, and scikit-learn. The Kannada workflow fixes the random seed at 42. Label encoders are serialized for reproducible label decoding, and task-specific best model checkpoints are retained. The repository is organized by subtask and contains the corresponding training, prediction, and README files.

GitHub repository: https://github.com/Harish-sri-ragunath-V/Innovators-DravidianCodeMix-2026

10. Discussion

The official results demonstrate that the multilingual Transformer foundation is effective across heterogeneous DravidianCodeMix settings, but they also show that task-specific design remains important. The best result for Innovators is the Kannada hate-speech system, while the highest rank is obtained in the Tulu task. Tamil-English achieves second place overall but exhibits a large performance imbalance across its five hierarchy levels.

The consistent top-three ranking across all three submissions is notable because the tasks differ in both linguistic setting and annotation structure. However, leaderboard rank should not be interpreted as evidence that the same architecture is optimal for every label space. The results instead support a flexible design in which a common multilingual encoder is adapted with specialized prediction heads and loss functions.

Future work should investigate transliteration-aware preprocessing, targeted data augmentation, task-specific learning-rate and sequence-length tuning, label-aware sampling, parameter-efficient fine-tuning, probability calibration, and ensemble methods. For the hierarchical Tamil-English task, modelling parent-child dependencies more explicitly may be especially useful. For Tulu, additional analysis of fine-grained label boundaries and class distributions could improve the joint objective.

11. Conclusion

This working note described the Innovators team's participation in all three DravidianCodeMix 2026 task settings using multilingual XLM-R-based classifiers. The systems adapt the Transformer representation to hierarchical Tamil-English hate speech, joint coarse/fine Tulu hope speech, and binary Kannada hate speech. The official leaderboard results place Innovators second, first, and third in the three subtasks, respectively. The experiments also reveal clear opportunities for improving deeper hierarchical labels and fine-grained hope-speech recognition. The released repository provides the training and prediction workflow needed to reproduce the reported approach.

12. Copyright Clause

© 2026 Copyright for this paper by its authors. Use permitted under Creative Commons License Attribution 4.0 International (CC BY 4.0).

The authors will complete and submit any separate copyright or author agreement required by the DravidianCodeMix 2026 and CEUR-WS organizers.

Acknowledgments

We thank the DravidianCodeMix 2026 organizers and the FIRE community for providing the shared-task resources and evaluation framework.

References

[1] DravidianCodeMix 2026 Organizers. DravidianCodeMix 2026: Shared Task on Hate and Hope Speech Detection in Dravidian Languages. FIRE 2026.

[2] Sripriya, N., Chakravarthi, B. R., Durairaj, T., Bharathi, B., Kumaresan, P. K., Subalalitha, C. N., Gowda, A. M. D., Hegde, P. R., and Vikram, D. Overview of the Shared Task on Offensive Language Identification in Dravidian Code-Mixed Languages. Working Notes of FIRE 2025, 2025, pp. 225–233.

[3] Chakravarthi, B. R., Priyadharshini, R., Muralidaran, V., Jose, N., Suryawanshi, S., Sherly, E., and McCrae, J. P. DravidianCodeMix: Sentiment Analysis and Offensive Language Identification Dataset for Dravidian Languages in Code-Mixed Text. Language Resources and Evaluation, 56, 765–806, 2022. https://doi.org/10.1007/s10579-022-09583-7.

[4] Conneau, A., Khandelwal, K., Goyal, N., Chaudhary, V., Wenzek, G., Guzmán, F., Grave, E., Ott, M., Zettlemoyer, L., and Stoyanov, V. Unsupervised Cross-lingual Representation Learning at Scale. Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, 2020, pp. 8440–8451.

[5] Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., Levy, O., Lewis, M., Zettlemoyer, L., and Stoyanov, V. RoBERTa: A Robustly Optimized BERT Pretraining Approach. arXiv:1907.11692, 2019.

[6] Loshchilov, I. and Hutter, F. Decoupled Weight Decay Regularization. International Conference on Learning Representations (ICLR), 2019.

[7] Chakravarthi, B. R., Muralidaran, V., Priyadharshini, R., Chinnaudayar Navaneethakrishnan, S., McCrae, J. P., García-Cumbreras, M. Á., Jiménez-Zafra, S. M., Valencia-García, R., Kumaresan, P. K., Ponnusamy, R., García-Baena, D., and García-Díaz, J. A. Overview of the Shared Task on Hope Speech Detection for Equality, Diversity, and Inclusion. Proceedings of the Second Workshop on Language Technology for Equality, Diversity and Inclusion, 2022, pp. 378–388.

[8] Chakravarthi, B. R. Multilingual Hope Speech Detection in English and Dravidian Languages. International Journal of Data Science and Analytics, 14, 389–406, 2022. https://doi.org/10.1007/s41060-022-00341-0.

[9] Achamaleh, T., Kawo, L., Batyrshini, I., and Sidorov, G. Tewodros@DravidianLangTech 2024: Hate Speech Recognition in Telugu Codemixed Text. Proceedings of the Fourth Workshop on Speech, Vision, and Language Technologies for Dravidian Languages, 2024.

[10] Zamir, M., Tash, M., Ahani, Z., Gelbukh, A., and Sidorov, G. Lidoma@DravidianLangTech 2024: Identifying Hate Speech in Telugu Code-Mixed: A BERT Multilingual. Proceedings of the Fourth Workshop on Speech, Vision, and Language Technologies for Dravidian Languages, 2024, pp. 101–106.

[11] Chava, S., Kumar, R., Saumya, S., and Biradar, S. IIITDWD_SVC@DravidianLangTech-2024: Breaking Language Barriers; Hate Speech Detection in Telugu-English Code-Mixed Text. Proceedings of the Fourth Workshop on Speech, Vision, and Language Technologies for Dravidian Languages, 2024, pp. 119–123.

[12] Lal, J. G., B., P., Chakravarthi, B. R., Rajiakodi, S., Bharathi, B., Natarajan, R., and Rajalakshmi, R. Overview of the Shared Task on Multimodal Hate Speech Detection in Dravidian Languages: DravidianLangTech@NAACL 2025. Proceedings of the Fifth Workshop on Speech, Vision, and Language Technologies for Dravidian Languages, 2025, pp. 114–122.

[13] Shanmugavadivel, K., Subramanian, M., Naveenram, C. E., Rs, V., and Srinesh, S. KEC_AI_ZEROWATTS@DravidianLangTech 2025: Multimodal Hate Speech Detection in Dravidian Languages. Proceedings of the Fifth Workshop on Speech, Vision, and Language Technologies for Dravidian Languages, 2025, pp. 232–236.

[14] Bhuvana, J., Mirnalinee, M. T. T., Rohan, R., Seshan, D., and Koushik, A. SSNTrio@DravidianLangTech 2025: Hybrid Approach for Hate Speech Detection in Dravidian Languages with Text and Audio Modalities. Proceedings of the Fifth Workshop on Speech, Vision, and Language Technologies for Dravidian Languages, 2025.

[15] Selvamurugan, A. DravLingua@DravidianLangTech 2025: Multimodal Hate Speech Detection in Dravidian Languages using Late Fusion of Muril and Wav2Vec Models. Proceedings of the Fifth Workshop on Speech, Vision, and Language Technologies for Dravidian Languages, 2025, pp. 694–699.